System Design Basics

Lesson 7: Reliability & Fault Tolerance

Reliability & Fault Tolerance - Study Notes

Key Concepts

1. **Replication** – Keeping multiple copies of data or services to improve availability and durability.
2. **Failover** – Automatic switching to a standby replica when the primary instance fails.
3. **Circuit Breaker Pattern** – Protects a system from cascading failures by halting calls to an unhealthy service.
4. **Graceful Degradation** – Designing systems to continue operating with reduced functionality when components fail.
5. **Health Checks & Monitoring** – Continuous probing that informs failover and circuit breaker decisions.

Summary

Reliability and fault tolerance are about ensuring that a system continues to deliver its core functionality even when parts of it break. The most common technique is **replication**, where data or service instances are duplicated across multiple nodes, zones, or regions. Replication can be synchronous (writes are confirmed on all replicas before returning) or asynchronous (writes are acknowledged on the primary first, then propagated later), each with its own trade offs between consistency and latency.

When a replica becomes unavailable, a **failover** mechanism detects the problem (usually via health checks) and routes traffic to a healthy standby. Failover can be *active passive* (one primary, one or more hot standbys) or *active active* (all replicas serve traffic simultaneously). Proper DNS, load balancers, or service meshes handle the routing, while state synchronization mechanisms keep the replicas consistent.

Even with replication and failover, downstream services can become overloaded or return errors, leading to **cascading failures**. The **circuit breaker pattern** mitigates this risk. A circuit breaker monitors the success/failure rate of calls to a remote service. If failures exceed a configurable threshold, the breaker “opens” and short circuits further calls, optionally returning a fallback response. After a cool down period, the breaker moves to a “half open” state to test if the service has recovered. This protects both the caller and the failing service from being hammered.

Together, these patterns create resilient architectures that can survive hardware outages, network partitions, and software bugs while still providing a usable experience to end users.

Important Points

- **Replication Types**
- *Synchronous*: strong consistency, higher latency, used for critical data (e.g., financial transactions).
- *Asynchronous*: eventual consistency, lower latency, suitable for analytics or cache layers.
- **Failover Strategies**
- *Active Passive*: simple, lower cost, but standby may become stale.

- *Active Active*: higher throughput, automatic load balancing, requires conflict resolution.
- *Cold Standby*: instance is started only after failure; longest recovery time.

• **Circuit Breaker States**

1. **Closed** – calls pass through; failures are counted.
2. **Open** – calls are blocked; immediate fallback returned.
3. **Half Open** – a limited number of test calls are allowed; success closes the breaker, failure re opens it.

• **Key Metrics for Health Checks**

- Latency (p99 response time)
- Error rate (HTTP 5xx, timeouts)
- Resource utilization (CPU, memory, thread pool saturation)

• **Graceful Degradation**

- Provide reduced feature responses (e.g., read only mode, cached data) instead of a total outage.

• **Idempotency**

- Crucial for retry logic in failover and circuit breaker scenarios to avoid duplicate side effects.

Examples

1. Replication + Failover with a Database (PostgreSQL)

```yaml
# Primary instance
primary:
  host: db-primary.example.com
  port: 5432
# Synchronous replicas (hot standby)
replicas:
  - host: db-replica-1.example.com
    port: 5432
  - host: db-replica-2.example.com
    port: 5432
```

• **How it works**:

1. Writes are sent to the primary. PostgreSQL's streaming replication copies WAL records to replicas synchronously.
2. A load balancer (e.g., PgBouncer, HAProxy) performs health checks on each node.
3. If the primary stops responding, the balancer promotes the most up to date replica to primary and redirects traffic.

• **Benefit**: Zero downtime read/write service for most failures; data loss limited to the last uncommitted transaction.

2. Circuit Breaker in a Microservice (Node.js + `opossum`)

```javascript
const CircuitBreaker = require('opossum');
const axios = require('axios');
function callPaymentService(order) {
  return axios.post('https://payment.example.com/charge', order);
}
// Configure breaker: open after 5 failures within 10 seconds, stay open 30 sec
const breaker = new CircuitBreaker(callPaymentService, {
  timeout: 3000,
  errorThresholdPercentage: 50,
  volumeThreshold: 5,
  resetTimeout: 30000
});
breaker.fallback(() => ({
  status: 'deferred',
  message: 'Payment will be processed later.'
}));
// Use the breaker
breaker.fire(order)
  .then(res => console.log('Payment ok:', res.data))
  .catch(err => console.error('Payment failed:', err));
```

• **What happens**:

- While the payment service is healthy, calls go through.

- If the service starts returning errors or timeouts, after 5 failures the breaker opens. Subsequent calls instantly receive the fallback response, preventing the order service from queuing up thousands of pending requests.

- After 30 /seconds the breaker tests the service; if it’s healthy, the circuit closes again.

Quick Review

1. **What is the main trade off between synchronous and asynchronous replication?**
2. **Name two failover architectures and a scenario where each is preferred.**
3. **Describe the three states of a circuit breaker and when it transitions between them.**
4. **Why is idempotency important when implementing retries in a fault tolerant system?**
5. **Give an example of graceful degradation you might implement in a web application.**

Further Reading

• **CAP Theorem** – Understanding consistency, availability, and partition tolerance trade offs.

• **Distributed Consensus Algorithms** – Raft, Paxos, and how they underpin leader election for failover.

• **Service Meshes** – Envoy/Istio built in health checks, retries, and circuit breaker policies.

- **Chaos Engineering** – Practices (e.g., Netflix's Chaos Monkey) for validating fault tolerance.
- **Eventual Consistency Patterns** – CRDTs, read repair, and anti entropy mechanisms.